clautolisp Debugger — Design Notes

Codex

June 24, 2026

Contents

1 Purpose

This document records the **design decisions** taken while implementing the clautolisp debugger — the rationale and the alternatives considered — so the choices are not re-litigated from scratch later. It complements:

- `clautolisp-debugger-spec.org` — the normative in-image debugger spec.
- `clautolisp-debugger-remote-cad-spec.org` — the remote-CAD addendum.
- `clautolisp-debugger-implementation-plan.org` — the phased plan (§3a is the interpreter two-bodies mechanism these notes elaborate).

Each entry is dated by the phase that introduced it and is written as a small ADR (context → decision → consequences).

2 DN-1. Poll points: `%CLAL-POLL` as a special operator (not a macro)

Phase 1. The instrumenter weaves a poll-point node around every evaluated compound form: `(%CLAL-POLL fid k inner)`. At evaluation it must fire `poll-point :before`, evaluate `inner` once, fire `poll-point :after`, and return `inner`'s value. The question was how to realise that node.

2.1 Architectural constraints

- clautolisp is a **tree-walking interpreter with no macro-expansion phase**. AutoLISP has no `defmacro`; `clautolisp.autolisp-runtime::*special-operator-dis` is the **only** special-form mechanism, and there is no `macroexpand` step.
- A poll-point wrapper must (1) take `fid=/=k` as **literals**, (2) evaluate `inner` **exactly once** in the right dynamic context, and (3) **return** `inner`'s value, bracketed by the before/after calls.
- Decisive: a **SUBR (function) receives evaluated arguments** (`call-autolisp-function-in-` maps `autolisp-eval` over the args before applying), so an ordinary function **cannot** receive an unevaluated `inner`. Only a special operator — or a compile-time rewrite — can.

2.2 Decision

Register `%CLAL-POLL` as a **special operator** in the interpreter's dispatch table (via the runtime's `register-special-operator`), with a handler that does `poll(before); v = (autolisp-eval inner); poll(after); v`. The woven body stays plain AutoLISP data run by the **same** evaluator.

2.3 Why not a macro (in the interpreter)

"Macro" has no native meaning in this interpreter; the only macro-like option is **instrument-time expansion into ordinary AutoLISP forms**. That is strictly worse here:

- There is no clean AutoLISP form that calls back into the host `poll-point`, evaluates `inner` once, *and* returns its value. `(progn (before) inner (after))` discards `inner`'s value; fixing it needs a `setq` temp — which **pollutes the dynamic binding stack** (the snapshot/inspector would show a fake variable, it can collide with user names, and it mutates a frame).
- The thunk alternative — wrap `inner` in a `lambda` and `funcall` it from a SUBR — **allocates a closure on the hot path** (against the allocation-free poll-path goal), adds a call frame, and the lambda body would itself need re-instrumentation guards.
- A macro that merely expands back into a special operator buys nothing.

2.4 Consequences

Accepted:

- **Pros.** Fits the existing evaluator with no change to the hot `autolisp-eval` loop; unevaluated operands + single evaluation + value return come for free; one representation (the instrumented body is plain AutoLISP data); all policy (volatile clearing, conditions, trace actions, `current-pp`) lives in one handler; trivially recognised as a leaf on re-instrumentation via `known-special-operator-p`.
- **Cons (accepted as minor).** `%CLAL-POLL` is a global, user-visible operator (the `%=/=clal-` naming discourages misuse); `special-operator-function` does an `assoc=/=string=` over the dispatch list for **every** cons operator everywhere, and the entry is `push`'d to the front, so every

other special-op lookup does one extra failed compare first — left as is (see below); each poll point pays one extra `autolisp-eval` dispatch and pushes a noise `(:eval . (%CLAL-POLL …))` frame onto `*autolisp-call-stack*` that the Phase-2 snapshot must filter.

Considered and deferred:

- **`append` vs `push`.** Appending the entry to the tail of `*special-operator-dispatch*` would avoid taxing common-operator lookups. Decision: **leave it as `push`** — the cost is negligible and not worth the churn until profiling says otherwise.
- **Dedicated AST struct node** (instead of a symbol-headed list). Pros: not a user-visible symbol, no collision, faster `typecase` dispatch. Cons: forces a change to the core `autolisp-eval` loop and the instrumented body would no longer be plain AutoLISP data — exactly what the special-operator approach avoids. Revisit only if frame-noise or dispatch cost shows up in profiling.

2.5 The compiler path is different (Phase 8)

This decision is scoped to the **interpreter**. When the Phase-8 compiler emits **CL** bodies, the idiomatic and best realisation is a **CL macro**:

```
(defmacro %clal-poll (fid k inner)
  `(progn (poll-point ,fid ,k :before)
          (multiple-value-prog1 ,inner (poll-point ,fid ,k :after))))
```

It expands inline, leaves no runtime operator, costs nothing at run time, and matches the spec §5.3 shape literally. So the two mechanisms are not competitors: the special operator serves **interpreted** bodies, a CL macro serves **compiled** bodies — two backends of the same instrumentation, exactly as the interpreter-now / compiler-later split (plan §3a, spec §13) intends.

3 DN-2. Two-body selection is per-function, not propagated along the call path

Phase 1, revising the original §3a propagation rule. The first cut of the two-bodies dispatch (`call-autolisp-function-in-context`) propagated

debugging-ness down the call chain: entering an *instrumented* body kept `*debugging*` set; entering an *uninstrumented* body cleared it for that subtree, so debugging-ness flowed only across the instrumented call sub-graph (the interpreter analog of Fjeld's debug entry points).

3.1 The flaw

With **selective** instrumentation, that rule is too restrictive. Consider an uninstrumented function `G` (a library function you have no source for, or a higher-order function) that calls `S`, one of *your* functions that you *do* want to debug. Under propagation, entering `G` cleared `*debugging*`, so the call `G → S` ran `S`'s plain body and breakpoints in `S` went silent. The only remedy was to instrument `G` too — impossible without its source.

Nuance on which intermediaries broke it (others are transparent):

Intermediary between you and S	`*debugging*` through it	S debugged?
SUBR / builtin (mapcar, apply)	passed through unchanged	yes
instrumented usubr	stays set	yes
uninstrumented usubr	**cleared**	**no** ← bug

3.2 Why propagation was the wrong import

Strict Fjeld propagation belongs to the **compiler** model (spec §5), where *every* function has both an ordinary and a debug body, so propagation is the only way to choose which runs. But:

- The spec's **interpreter** variant (§11.3, §13 v1) is just a per-thread `*debug-mode*` flag — no propagation.
- The **remote** model installs `S~dbg` under the public name `S` (R-II.2), so an instrumented function is reached in its debug form **even when called by native/library code**; uninstrumented callees (REQ-LOAD2) simply run native.

So propagation made the interpreter **more** restrictive than both the spec's interpreter variant and the remote backend. It was the wrong half of the compiler design to pull forward.

3.3 Decision

Body selection is **per function**, gated only by whether a debug session is active on this thread:

```
(if (and *debugging* (autolisp-usubr-instrumented-body function))
    (autolisp-usubr-instrumented-body function)   ; instrumented  debug it
    (autolisp-usubr-body function))                ; else run plain
```

`*debugging*` is set once by the session entry (`call-with-debugging` / `run-debugged-thread`) and is **no longer rebound per call**. It means simply "a debug session is active on this thread"; it is the runtime-side twin of the debugger's per-thread `debug-flag` (the runtime cannot see `clautolisp.debug`, so it carries its own session flag, and the session entry sets both).

3.4 Consequences

- **Instrument exactly what you want to debug; it is debugged wherever it is called from** — your code, an uninstrumented library `G`, a HOF, or a builtin. This is the interpreter twin of the remote public-name swap.
- **Uninstrumented functions always run plain** — zero overhead, no poll points, not stoppable (instrument them, with source, to debug into them).
- Per-thread isolation (spec §23) is preserved: `*debugging*` is bound per debugged thread, so other threads (and the debugger thread running, say, the printer) still run plain bodies — no global-breakpoint deadlock.
- Locked in by the test `instrumented-callee-debugged-through-uninstrumented-caller`.

3.5 Trade-off given up (and the recovery)

Propagation had one side benefit: "debug `S` *only* when reached through instrumented code", letting you keep `S` un-debugged on a hot path `P` while debugging it via `F`. Per-function selection cannot express that by choice of *what to instrument* alone — an instrumented `S` stops on every path. Recover call-context sensitivity with a **conditional breakpoint** (the condition inspects the caller / a variable), which is the right tool for it anyway. If a project ever

needs the old behaviour wholesale, a per-session `propagate-debugging-p` flag (default off) could re-enable it; not added preemptively.

3.6 Compiler path (Phase 8)

Unchanged from DN-1's note: the compiler still emits two CL bodies, but the "which body runs" decision should likewise be driven by whether a function was compiled-for-debug and a session is active, installed the remote way (debug body under the public name), rather than by call-path propagation.

4 DN-3. Phase-2 stepping is depth-based; precise stop points

Phase 2. Stepping (spec §6) is implemented as the degenerate volatile breakpoint: a pending *step request* that the poll-point routine tests against two nesting depths that `eval-poll-form` maintains while a session is active on the thread (kept balanced on non-local exit by `unwind-protect`):

- **poll-depth** — form nesting: +1 at each form's `:before`, −1 at its `:after`.
- **call-stack** — function nesting: a shadow frame pushed at each function-entry poll point (form-id 0) and popped at its exit. Its length is the call depth; it also backs the snapshot's `call-stack` (§9.3).

Why depths and not `parent-form-map`: poll points are already perfectly nested (every `%CLAL-POLL` brackets a form, and a called function's poll points nest inside the call form's `:before=/`:after=), so a single integer captures "am I inside the thing I'm stepping over." `parent-form-map` remains in the metadata for future use (e.g. structural "step to end of enclosing form") but is not needed for in/over/out.

4.1 Semantics (depths captured when the step is armed: D0F form, D0C call)

- **step into** — stop at the next form beginning (`:before`, any depth): descend into the first sub-form, or into a called function's entry. If the called function is *uninstrumented* it has no poll points, so "into" naturally degrades to "over" it.

- **step over** — stop at the next `:before` with poll-depth D0F (skipping the current form's sub-forms and any function it calls), or as soon as the current function returns (call-depth < D0C).
- **step out / finish** — run until the current function returns (call-depth < D0C).

4.2 A non-obvious but correct stop point

Step out lands on the call form's `:after`, not the next statement's `:before`. Stepping out of `ID` called from `(setq z (id x))` stops at the `:after` poll point of the `(id x)` form in the caller — i.e. *just after the call returns*, value in hand — which is the natural "I'm back in the caller" point. The next statement's `:before` is a *further* step. Likewise, atom statements/operands carry no poll point (only compound forms are wrapped), so "the next statement" is only a stop point when that statement is itself a compound form. Tests pin these exact stop points so the semantics can't drift silently.

4.3 Re-entrancy of eval-in-frame

`eval-in-frame` (and any debugger-side evaluation) runs with `*debugging*` rebound to NIL, so it selects plain bodies and cannot re-enter the debugger through its own poll points (the analog of the host debugger running with debugging off). Phase 2 supports the innermost frame; outer-frame evaluation (`with-frame-bindings`, §9.3) is deferred (the spec rates it Partial/risky even remotely).

4.4 Deferred to Phase 3 (slots present, empty)

The snapshot carries `globals-touched` (§9.6, needs instrumented global read/write capture) and `catch-stack` (§10.2, needs `vl-catch-all-apply` integration) as empty lists in Phase 2; they are populated when the §10 error integration lands.

5 DN-4. Error integration via HANDLER-BIND; the unhandled/caught split is free

Phase 3 (spec §10). An AutoLISP error in clautolisp is a CL `autolisp-runtime-error` raised by `signal-autolisp-runtime-error`. The debugger must stop *at*

the error point with the live stack intact, so it intercepts with `HANDLER-BIND` (whose handler runs before the stack unwinds), not `HANDLER-CASE` (which unwinds first). `debug-handle-error` builds the snapshot from `current-pp` (the innermost poll point = the erroring form), dispatches it to `*debug-hit-handler*`, and performs the §10.1 resolution.

5.1 The §10.1 / §10.2 split falls out of handler ordering — for free

`vl-catch-all-apply` already wraps its applied function in a `HANDLER-CASE` for `autolisp-runtime-error`. The session's `HANDLER-BIND` is established *outside* the application (hence outside any `vl-catch-all-apply` within it). CL runs handlers innermost-first, and a `HANDLER-CASE` handler transfers control the moment it runs. So:

- error inside a `vl-catch-all-apply` → the inner `HANDLER-CASE` fires first and transfers control; the session `HANDLER-BIND` never runs *caught* (§10.2).
- error not inside any `vl-catch-all-apply` → only the session `HANDLER-BIND` applies *unhandled* (§10.1).

No bookkeeping distinguishes them; the condition system does.

5.2 Resolutions (§10.1)

- **continue-with-error** (default; also `:continue` / NIL) — the handler returns normally, *declining*, so the condition keeps propagating to the user's `*error*` exactly as without the debugger (`call-with-autolisp-error-handler` → the `*ERROR*` function).
- **abort** — `(throw 'clal-abort :aborted)` to a `CATCH` the session establishes; `call-with-debugging` returns `:aborted`, the threaded run reports `(:aborted)`. (Restoring system variables is the user `*error*`'s job; the debugger's abort does not attempt it.)
- **continue-with-return VALUE** — each poll point's form evaluation is wrapped in a `CLAL-POLL-RETURN` restart; the handler `invoke-restart`'s the innermost one, so the **innermost instrumented form enclosing the error** returns VALUE. Safe only for form-internal errors; with no enclosing instrumented form there is no restart and we decline.

5.3 §10.2 catch frames and break-on-caught

The `vl-catch-all-apply` builtin binds a runtime `*autolisp-catch-stack*` (list of `catch-frame`) around the applied function, so any snapshot taken inside the catch shows it (§9.2 `catch-stack`). The builtin also calls a runtime hook `*autolisp-caught-error-hook*` when set; the session installs it only under `:break-on-caught` (default off). Same inversion as `%CLAL-POLL` (DN-1): the runtime/builtins expose a hook, the debug system fills it — so `autolisp-builtins-core` (which owns `vl-catch-all-apply`) needs no dependency on `clautolisp.debug`. The catch-stack/hook variables live in `clautolisp.autolisp-runtime` for exactly that reason.

5.4 Deferred

- `globals-touched` (§9.6) still empty — needs instrumented global read/write capture; folds in with a later pass.
- The break-on-caught snapshot is taken at the catch point (the `HANDLER-CASE` has already unwound the deep frames). Capturing the deep stack would require converting `vl-catch-all-apply` to `HANDLER-BIND`; deferred as not worth the churn for v1.

6 DN-5. The inspector is a runtime-only library; values computed in CL, accessors are data

Phase 4 (spec Part IV). The inspector (`clautolisp.inspect`) is "a separate library from the debugger" (§14): the debugger *uses* it, so `clautolisp.debug` must be able to depend on it — which means the inspector must NOT depend on the debugger (no cycle). It therefore depends on the **runtime only**.

6.1 Two-level value access: compute in CL, keep accessors as data

Each page component carries an **accessor** — an AutoLISP S-expression template with the free placeholder `_` (e.g. `(cdr (assoc 10 (entget _)))`). Crucially:

- the component's **value** is, where possible, computed **directly in CL** — a runtime cons is a CL cons, so `car=/=cdr=/=nth` need no evaluator; string chars, a symbol's value/function (via lookup) likewise. So

inspecting numbers / strings / symbols / lists needs **no builtins and no host**.

- the **accessor** is kept purely as **data**, used only to compose the path expression (§15.2). Descent uses the already-computed component value, never re-evaluation. So navigation and path composition are evaluator-free too.

Only the host-dependent types (`ename`, `pickset`) must evaluate their accessor (`(entget _)`, `(ssname _ n)`) to enumerate components; they do so via `autolisp-eval` in the session's context and **degrade gracefully** (a single `:unrealized` component) when no host is present. This is why the test suite can cover the ename group-code path — it stubs `ENTGET` as a user function returning a canned DXF list, with no host at all.

6.2 Path expressions by substitution (§15.2)

`session-path-expression` folds the history's accessor chain from the origin, `(subst expr '_ accessor)` at each step (the placeholder `_` is the interned AutoLISP symbol "_", matched by `EQ`). An `:opaque` step (a value with no AutoLISP-expressible accessor) stops the fold and returns `(values partial :partial)`. Opaque components stay **descendable** (you can still look), per §15.1.

6.3 Workspace and the name resolution boundary (§16.2)

`$N` slots and the `$0=/=*` aliases live in the inspector's own `workspace`, resolved **inspector-side** by `session-eval` (which binds them in a temporary dynamic frame, with `*debugging*` off so the eval can't re-enter the debugger) — never by the AutoLISP evaluator, so they never intrude on the AutoLISP symbol table. `session-bind` destinations: `:workspace` is pure inspector state; `:global=/`:setq= write the runtime namespace/binding; `:frame` is delegated to a **pluggable writer** the debugger installs (`bind-frame-fn`), so the inspector needs no knowledge of debug snapshot frames — the debugger wires its `set-binding-entry` in when it opens an inspector on a binding (§16.1).

6.4 Note

`clautolisp.inspect` shadows `cl:inspect` (it defines its own `inspect`, the §14 session entry).

6.5 Deferred

- VLA-object property enumeration needs ActiveX; the page shows a preview and defers descent to the REPL (`(vla-get-PROP _)`). Hash-table pages are not provided (no core AutoLISP hash type).

7 DN-6. UI protocol: synchronous directive-returning command loop for in-image UIs

Phase 5 (spec §17, §18, §21–§24). The debugger is UI-agnostic: a UI is a CLOS object implementing `ui-*` notifications (NIL-default generics, so a UI implements only what it needs, §17.1) plus one value-returning generic, `ui-await-command`, which drives the UI's command loop and returns a **resume directive**. The engine's existing `*debug-hit-handler*` boundary (Phase 1) is exactly where a session plugs in: `call-with-session` binds it to a closure that records the snapshot, fires the right notification, runs `ui-await-command`, and returns its directive.

7.1 Why synchronous (not the two-thread queue) for terminal UIs

The parent spec frames §8 around a two-thread pause (app thread blocks on a queue; debugger thread drives). That path already exists (`run-debugged-thread` / `continue-thread` / `step-thread`, Phase 1–3) and is what the **Emacs** UI will use over its RPC. But for in-image terminal UIs (dumb, ncurses) the UI runs **in the application thread itself**, inside `*debug-hit-handler*`: `ui-await-command` reads commands and returns a directive that the engine applies in place. This is simpler, needs no second thread, and is what §17.2's "ordinary CL calls — the UI is in-image for terminal UIs" describes. The two models share the same directive vocabulary (`:continue` | `(:step KIND)` | `(:advance …)` | `:abort` | `(:continue-with-return V)`), so the engine is identical underneath; only who calls `ui-await-command` (the app thread vs an RPC worker) differs.

7.2 cmd-* return directives; the loop decides when to resume

Each `cmd-*` that resumes (`cmd-continue`, `cmd-step`, `cmd-abort`, `cmd-return`, `cmd-advance`) **returns** the directive; non-resuming commands (set a breakpoint, eval, inspect, print) return NIL and the dumb UI's loop keeps reading.

So the loop is just "read a line → dispatch → if it returned a directive, resume." This keeps the resume decision in one place and makes the command set trivially extensible.

7.3 The inspector is wired in, not duplicated

`cmd-inspect` opens a `clautolisp.inspect` session sharing the **debugger workspace** (so `$N` persists across stop points, §16.2) and installs the `:frame` binding writer (DN-5's pluggable `bind-frame-fn`) that maps to `set-binding-entry` on the snapshot — the one place the inspector needs debugger-specific behaviour, supplied from outside so the inspector library stays debugger-free.

7.4 Streams are injected; EOF means continue

The dumb UI (§18) reads/writes injectable streams (default `*standard-input*=/=*standard-output*` so the whole UI is testable over string streams — the suite drives full break→inspect→step→continue sessions and asserts on captured output. EOF at the prompt returns `:continue` (so a debugger left in a pipe or CI run never blocks, §18.3).

7.5 Package note

The dumb UI `:use=s =clautolisp.debug.ui` and selectively imports the inspector `session-*` it calls directly (`session-page=/=session-origin=/=session-eval`); the engine's `session-snapshot=/=session-inspector` (debugger-session accessors) and the inspector's `session-*` live in different packages and don't collide because the UI protocol package deliberately does not re-export the inspector's `session-*` symbols it imported for internal use.

8 DN-7. ncurses UI: a curses-free screen abstraction, tested via a mock grid

Phase 6 (spec §19). The four-pane ncurses UI must run on a real terminal yet be unit-testable headlessly. The spec anticipates this with §19.3's `clautolisp.ui.tui` abstraction. The split:

- `clautolisp.ui.tui` — a tiny screen protocol (`tui-start/stop/size/clear/put/refresh/read-key`), a pane/layout model (`four-pane-layout`, `draw-box`, `pane-put-line`), and a **mock-screen** backend recording a

character grid + per-cell attribute and replaying a scripted key queue. **No curses dependency**, so it is in the aggregate and CI.

- `clautolisp.ui.ncurses` — the §19 four-pane UI (stack TL, source TR, interactor BL, repl BR) implementing `clautolisp.debug.ui` **entirely against the abstraction** — it never calls curses. `ui-await-command` is a key-driven event loop; output also accrues into UI slots (`message`, `repl-lines`, `selected-frame`) so tests assert on state, not only the grid. Driven through the mock screen, the suite covers continue/step, frame navigation, breakpoint toggle at the cursor line, eval into the repl, the inspector pane, abort, and EOF — all headless.

- `clautolisp/autolisp-debug-ui-tui-charms` — the real cl-charms backend, in a **separate .asd outside clautolisp.asd** and deliberately **not in the aggregate or its tests**: it needs cl-charms/libncurses and a real terminal. It implements the same screen protocol, so the UI and every test are identical whether the backend is the mock or charms.

8.1 Source markers without curses

The source pane colours each line via the abstraction's ATTR keyword (:yellow current, :red breakpointed poll-point line, :blue plain poll-point line, §19.1). The mock records the attr per cell, so a test asserts e.g. "the `>>` column on the current line is :yellow" — colour verified with zero terminal.

8.2 Package note

`clautolisp.ui.tui` and `clautolisp.debug.ui` both export `PANE`, so the ncurses package `:use=s =clautolisp.debug.ui` but selectively `:import-from=s the tui symbols it needs rather than =:use=ing both (which would clash). It also shadow-imports =clautolisp.inspect:inspect` over `cl:inspect`.

8.3 Deferred

PDCurses/Windows backend (same protocol, future sibling); live mouse and resize beyond the `:resize` key (key-only navigation is complete per §19.3).

9 DN-8. Inspector path composition must not use CL:SUBST (shared structure → cycles)

Phase 6 bug, fixing Phase 4. `session-path-expression` (§15.2) and the ename/pickset accessor evaluation originally folded accessors with `(subst expr '_ accessor :test #'eq)`. CL:SUBST is permitted to **share structure with its TREE argument**, and because every page-built accessor embeds the **same interned placeholder symbol `_`**, that sharing spliced the accumulating expression into shared accessor structure and produced a **circular** path expression (printing as `#1=(CDR (CAR #1#))`) plus a spurious `:partial`. The Phase-4 cons test missed it because its particular descents happened not to alias; the ncurses inspector-pane test (descend then read the path) surfaced it.

Fix: a dedicated `substitute-placeholder` that **always conses fresh structure** on the spine and matches the placeholder by NAME (`placeholder-p`), so the result can never alias the accessor or the expression. Both `session-path-expression` and `%eval-accessor` use it; CL:SUBST is no longer used for path composition. Lesson: never compose reusable templates that share a sentinel symbol with CL:SUBST — substitute with explicit fresh consing.

10 DN-9. Abort works from any stop, not only an error stop

Phase 6 bug, fixing Phase 3. The `:abort` resume directive was honoured only on the §10.1 error path (`apply-error-directive` threw `clal-abort`); the normal-stop path (`apply-resume-directive`, for breakpoint and step stops) let `:abort` fall through to a no-op, so pressing `q=/=a` at an ordinary breakpoint silently **continued** instead of aborting. The ncurses `abort-key-aborts` test surfaced it (result `7`, not `:aborted`).

Fix: `apply-resume-directive` now also handles `:abort` by clearing any pending step and `(throw 'clal-abort :aborted)` — the same catch `call-with-error-integration` already establishes around all debugged execution. So a user can abort from **any** stop (breakpoint, step, or error), which is what every UI's `q=/=a` key implies. The dumb-UI and engine suites confirm no regression.

11 DN-10. The Emacs UI is an S-expression RPC shim; the wire is data, not code

Phase 7 (spec §20). The Emacs UI (`aldb`) splits cleanly in two:

- **`clautolisp.ui.emacs` (CL)** — a thin shim implementing the `clautolisp.debug.ui` protocol by writing each notification as ONE readable S-expression per line to an OUTPUT stream and reading command forms back from an INPUT stream (§20.1: line-oriented sexps, no length prefixes, no encoding negotiation). Both streams are injectable, so the whole shim is unit-tested **headlessly over string streams** — 27 checks covering attach/version, the serialized hit snapshot, eval, step, abort, set-breakpoint+list, select-frame, inspector descend/path, continue-with-return on an error, and EOF.
- **`aldb.el` (Emacs)** — the SLDB-modelled minor mode (§20.2 buffers, §20.3 keybindings) that reads those forms with `read` and writes commands back. It is shipped under the system's `emacs/` dir and **byte-compiles clean** under Emacs 30, but is NOT in the CL aggregate (it has no CL test surface — its contract is the wire, which the shim tests pin).

11.1 Everything on the wire is data

A subtle but load-bearing rule: the shim never sends a form the Emacs side would **evaluate as code**. Values, bindings, paths, and previews are sent as **strings** (`preview`), positions/frames/pages as plain tagged lists of strings+integers (`snapshot->wire`, `page->wire`). The only place user text becomes code is inside the CL side, where `:eval=/`:inspect=/=:return= command arguments are read with the AutoLISP reader and evaluated in the snapshot (via the existing `cmd-eval`). So a malicious or malformed value preview can never execute in Emacs, and the transport stays a dumb pipe — which is what lets it ride any existing CL Emacs channel (a swank socket, an inferior-lisp pipe; §20.1 leaves the transport to the user).

11.2 Reuse of the engine; no new debugger surface

`aldb` adds zero capability to the engine: every command maps to an existing `cmd-*` (§17.2), and the resume directives are the same `:continue` / `(:step …)` / `:abort` / `(:continue-with-return …)` the dumb and ncurses UIs return. Notably it inherits DN-9 (abort from any stop) for free. "A

feature missing in the dumb terminal is missing in Emacs" (§20.4) holds by construction.

11.3 Transport pluggability

`aldb-connect` takes a live process and wires a process-filter that buffers partial output and dispatches each complete top-level form. The CL shim is transport-agnostic too (just two streams). Neither side mandates sockets vs pipes vs a reused swank connection (§20.1).